

Mangalist — Product Catalogue: Architecture & Design

Display / browsing catalogue — design deliverable, no implementation performed by this document.

Date	2026-07-05
Authors	Solution Architect (this document) + DBA (data model, reproduced in full below) — both CIO-directed, reporting to the CIO
Status	DESIGN / RECOMMENDATION ONLY. Nothing in this document is approved, implemented, migrated, or deployed. All architecture deviations and any cost implications are explicitly flagged and require CIO → CEO → parent sign-off before Dev/DBA implement anything.
In scope	Categories & browsing Search & filtering Product detail pages Images Variants (size/color/model) Stock / availability display
Out of scope	Cart Checkout Payment Order flow — all deferred per <code>ceo/strategy.md</code> ; not designed here.

1. Overview

This document defines the technical design for a display/browsing-only product catalogue for Mangalist, at the business's actual current scale: two brands at launch (Char-Broil, Napoleon), roughly a couple of dozen SKUs, a two-person operations team (the twins), no orders yet, and no meaningful site traffic yet. The design deliberately does not build for hypothetical future scale — every place this document adds structure ahead of strictly proven need is called out explicitly, matching the DBA's own proportionality discipline in the schema design below.

The catalogue builds on the existing self-hosted Supabase stack (Postgres + PostgREST + RLS + Studio, run via Docker — locally today, with AWS EC2 as the already-agreed future deploy target) and the existing plain HTML/CSS/JS frontend (no framework, no build step, Hebrew/RTL). Architecture for this specific catalogue was explicitly reopened for reconsideration by the parent; this document evaluates alternatives and recommends, in almost every dimension, staying close to the existing pattern — with one genuine decision point (SEO / link-preview rendering for product pages) presented as an explicit either/or for the CIO/parent, not decided here.

2. Proposed Architecture

2.1 Frontend rendering & serving approach

Recommendation: keep the existing pattern — plain HTML/CSS/JS, client-side fetch from PostgREST, no framework, no build step.

Concretely: two new plain HTML pages, styled with the existing `styles.css` theme, sit alongside `index.html`:

- `catalog.html` — browse/filter/search listing page. State (selected category, brand, price range, stock filter, search text, sort order) lives in the URL query string (e.g. `catalog.html?category=gas-grills&brand=char-broil&stock=in_stock&q=...`), so results are linkable/shareable and the page needs no server-side routing.
- `product.html` — single product detail page, driven by an id in the query string (e.g. `product.html?id=<uuid>` — see §2.2 on why id/sku rather than a Hebrew slug for now).

Both pages use vanilla JS plus the `@supabase/supabase-js` CDN client already wired into `index.html` for the leads form, calling PostgREST directly with the public anon key. This is safe because RLS already restricts anon reads to `published = true` rows (same pattern as the existing `products` table policy) — no new trust boundary is introduced.

Rendering options considered

Option	Description	Verdict
A. Client-side fetch (recommended)	Static HTML shell + vanilla JS fetches from PostgREST at page load, renders into the DOM. Zero new tooling, identical deploy story to today (static files + self-hosted Supabase).	Recommended default. Proportionate to current traffic (near zero) and team size.
B. Build-time static pre-rendering	A small Node.js generator script (not a framework — no bundler, no client hydration library) queries Postgres/PostgREST at build time and emits one static HTML file per published product, with real <code><title></code> /meta description/Open Graph tags baked in, run manually or on a schedule whenever the catalogue changes.	Not recommended today — documented for later. See flag below. Real benefit: correct Google indexing and rich WhatsApp/Facebook link previews for individual product pages (client-side-only rendering means crawlers that don't execute JS — notably link-preview bots — see an empty page). Real cost: introduces a build step where none exists today, and a "someone must remember to regenerate" operational step.
C. Frontend framework (Next.js / Astro / Gatsby, SSR or SSG)	Full framework with routing, data-fetching conventions, build pipeline.	Rejected as over-engineering at this scale — a couple dozen SKUs, two brands, no orders, minimal traffic. Would add a real ongoing maintenance/learning-curve cost with no proportionate benefit over Option A or B.

RECOMMENDATION REQUIRING SIGN-OFF

This design recommends **Option A** (no build step, matching the existing "no framework, no build step" principle) as the default for now, since there is no organic-search or social-sharing traffic yet to justify the extra moving part. **Option B (build-time pre-rendering) is flagged as a genuine future decision point**, not a rejected idea — once Marketing has an active plan to drive traffic to specific product links (SEO, WhatsApp/Facebook sharing), the CIO/parent should revisit whether the SEO/link-preview benefit now justifies the small added build step. No cost is attached to either option (no paid service), but Option B is a deviation from the current "zero build step" architecture and needs explicit sign-off before Dev implements it, whenever it is revisited.

2.2 URL / routing scheme (partially resolves DBA's Hebrew-slug open question)

The DBA flagged that "slug" is an English SEO convention that doesn't map cleanly onto Hebrew product names (open question 1, §4). This design proposes a split resolution, since brands/categories and products have very different slug-ability:

- **Brands and categories** — small, curated, controlled lists (2 brands, a handful of categories). Recommend the twins/CIO assign real transliterated/English slugs (e.g. `char-broil`, `napoleon`, `gas-grills`, `accessories`) — low effort, done once, gives clean readable category/brand URLs today.
- **Products and variants** — dozens of Hebrew product names with no obvious transliteration convention yet. Recommend routing by `id` (or `sku` for variants) for now (`product.html?id=...`), leaving the `products.slug` column null/unused until Marketing settles a Hebrew product-URL convention. This defers a real decision to the people who own site copy/positioning, rather than the Solution Architect guessing at it.

2.3 Search & filtering — how it's served to the browser

Per the DBA's recommendation, search and faceted filtering are both served directly by Postgres — no separate search service. The catalogue page's filters map directly onto PostgREST query parameters against the `products` / `product_variants` views:

```
GET /rest/v1/products?select=id,name,slug,specs,brand:brands(name,slug),
  product_variants(id,sku,variant_name,price_ils,stock_status)
  &category_id=eq.<uuid>
  &brand_id=eq.<uuid>
  &product_variants.stock_status=eq.in_stock
  &fts_column=fts(simple).?>
  &published=eq.true
  &order=name.asc
```

Free-text search uses the DBA's generated `tsvector` column (`'simple'` config — Hebrew does not benefit from English stemming) via PostgREST's `fts` filter, exactly as recommended by the DBA. Facet filters (category, brand, price range, stock status) are plain equality/range filters using the existing indexes (`category_id`, `brand_id`, `price_ils`, `stock_status`). Facet *counts* (e.g. "Char-Broil (5)" badges next to filter checkboxes) are computed client-side from the already-fetched result set at this catalogue size — no materialized view or extra aggregation query is needed on day one, per the DBA's note that a materialized view is a cheap future addition only if heavy joined-aggregation queries become necessary later.

2.4 Product variants & stock/availability display

- A product with exactly one variant (the common case for single-SKU accessories) shows no variant picker — price/stock come straight from that one `product_variants` row.
- A product with >1 variant (e.g. a grill available in two sizes, or an accessory in multiple colors) shows a simple picker (dropdown or pill buttons) built from `variant_name` ; selecting a variant updates the displayed price, stock badge, dimensions, and swaps in variant-specific images where present (`product_images.variant_id`), falling back to the product-level gallery otherwise.
- Stock status badge, mapped from the DBA's `stock_status` enum to Hebrew UI labels:

Stock status → UI treatment

stock_status value	Suggested Hebrew label	Suggested color
<code>in_stock</code>	זמין להזמנה	Green
<code>out_of_stock</code>	אול מהמלאי	Gray/red
<code>preorder</code>	בהזמנה מראש	Amber
<code>discontinued</code>	הופסק	Dark red
<code>unknown</code> (default)	זמינות לפי בירור	Neutral gray

NOTE FOR MARKETING (not this document's call) Per `ceo/strategy.md` , the fulfillment model under consideration is order-on-demand from suppliers, not held warehouse inventory. "זמין להזמנה" (available to order) is suggested above instead of a literal "in stock" translation so the copy doesn't imply warehouse stock that doesn't exist. Final wording is Marketing's call, not decided here.

Price display: if `price_ils` is null (expected at launch — no real ILS pricing exists yet per the DBA schema), show "מחיר לפי פנייה" (price on request) linking into the existing contact form, consistent with the site's current contact-driven approach rather than inventing a placeholder price.

2.5 Images — handling & serving

Recommendation: use the Storage service already included in the self-hosted Supabase stack (Postgres + PostgREST + Studio + Auth + **Storage** — confirmed already part of what `supabase start` brings up per `supabase-local/README.md`). A single public-read `product-images` bucket, with upload restricted to the postgres/service role (same bypass-RLS pattern already used for table writes via Studio). Twins/CIO upload product photos directly through Studio's Storage UI (drag-and-drop); the resulting public URL is stored in `product_images.url` .

Image storage options considered

Option	Verdict
Supabase Storage (self-hosted, existing stack)	Recommended. No new component, no new cost, self-serve for the twins (matches the "Studio as CMS" approach already committed to for catalog rows), same Docker-to-EC2 deployment path as everything else.
Static image files committed to the git repo	Rejected — dozens of product photos bloat the repo over time and require a developer to add every image; doesn't give the twins self-serve upload.
External image CDN (Cloudinary, imgix, etc.)	Rejected for now — a new paid service with no proportionate benefit at near-zero current traffic. Flagged as a possible future revisit only if page-weight/performance becomes a real problem.

OPERATIONAL NOTE, NOT A COST The self-hosted stack has no automatic image optimization/transformation or CDN edge caching. Recommend the twins/Marketing resize photos to a reasonable web size (e.g. ~1200px wide) before uploading, as a manual practice — not a system requirement, no tooling to build.

COST FLAG (informational only, no new spend) Using Supabase Storage introduces no new paid service — it is already part of the self-hosted stack the parent previously approved standing up locally. The only cost implication is indirect: stored images add to the eventual AWS EC2 disk usage, which is already accounted for in the CIO's existing 60–150 NIS/month estimate for the single-instance deploy. Flagging for completeness, not because it's a new decision.

2.6 Deployment topology

Unchanged from the CIO's existing Phase 1 plan in `ceo/strategy.md`: the same self-hosted Supabase docker-compose stack (now with more tables and the Storage bucket in use) runs locally via Docker Desktop today, and is deployed as the same compose file to a single AWS EC2 instance later. The catalogue frontend pages (`catalog.html`, `product.html`) are additional static files served the same way `index.html` is today — no new hosting component, no new topology decision introduced by this catalogue design.

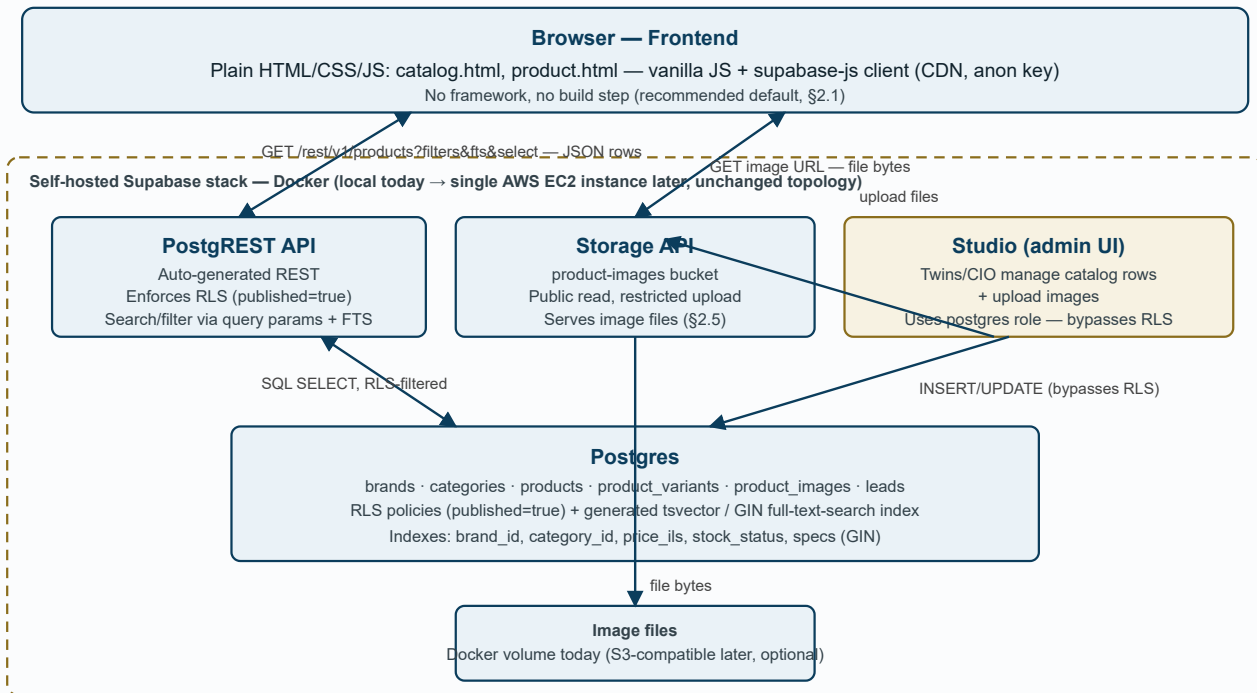


Diagram (a): system architecture and data flow — from browser request through PostgREST/Storage to Postgres and back, plus Studio's admin path used by the twins/CIO to manage catalog content and images.

3. Data Model (DBA design, reproduced in full)

The following is the DBA's authoritative catalog schema design, reproduced here so this document is self-contained for the CIO/parent. This Solution Architect design builds directly on it without alteration — see §2 above for how the frontend/API layer consumes it.

3.1 Entities

brands

id	uuid, PK
name	text, unique, not null
slug	text, unique, not null
logo_url	text, nullable
published	boolean, default false
created_at / updated_at	timestamps

categories

id	uuid, PK
parent_id	uuid, FK → categories(id), on delete restrict, nullable — self-reference tree; null = top-level (e.g. "Grills"/"Accessories")
name	text, not null
slug	text, unique, not null
sort_order	int, default 0
published	boolean, default false
created_at / updated_at	timestamps
Check constraint	parent_id is null or parent_id <> id — shallow cycle guard, acceptable given a 2-level hierarchy (e.g. Grills > Gas Grills, Accessories > Covers)

products (conceptual / model-line entity)

id	uuid, PK
brand_id	uuid, not null, FK → brands(id), on delete restrict
category_id	uuid, not null, FK → categories(id), on delete restrict
name	text, not null
slug	text, unique, nullable — Hebrew slug question flagged, see open questions and §2.2
description	text — long-form copy
specs	jsonb, not null, default '{}' — flexible spec bag (e.g. {"burners":4,"btu":36000,"primary_cooking_area_cm2":5800,"material":"cast iron"}), chosen over rigid columns since grills vs. accessories have very different attributes. Tradeoff: no DB-enforced key schema, needs an app-level key-naming convention.
published	boolean, default false
created_at / updated_at	timestamps

product_variants (the purchasable/sellable unit — price/stock attach here, not on products)

id	uuid, PK
product_id	uuid, not null, FK → products(id), on delete cascade
sku	text, unique, not null
variant_name	text, nullable (e.g. "Black", "Large 150x70x120cm"; null/"Default" if no real variation)
dimensions	jsonb, nullable — structured (e.g. {"width_cm":132,"depth_cm":61,"height_cm":114}) instead of today's free-text, so it's filterable/sortable
price_ils	numeric(10,2), nullable (no real ILS pricing exists yet); check price_ils >= 0 or null
stock_status	text, not null, default 'unknown'; check in ('in_stock', 'out_of_stock', 'preorder', 'discontinued', 'unknown')
stock_qty	integer, nullable — optional, not required given the order-on-demand fulfillment model
published	boolean, default false — own flag; read policy requires this AND parent product published (see RLS)
sort_order	int, default 0
created_at / updated_at	timestamps

product_images

id	uuid, PK
product_id	uuid, not null, FK → products(id), on delete cascade
variant_id	uuid, FK → product_variants(id), on delete cascade, nullable — set only for variant-specific shots (e.g. a color)
url	text, not null
alt_text	text — Hebrew accessibility/SEO
sort_order	int, default 0
is_primary	boolean, default false — partial unique index on product_images(product_id) where is_primary (at most one primary image per product)
created_at / updated_at	timestamps
Visibility	No own published column — visibility fully inherited from product (and variant if variant-scoped)

3.2 Relationships

- categories.parent_id — self-referencing tree (2 levels in practice today)
- products.brand_id / products.category_id — many-to-one
- product_variants.product_id — one-to-many (every product has ≥1 variant; a trivial "Default" variant covers single-SKU products)
- product_images.product_id — one-to-many
- product_images.variant_id — optional many-to-one
- Delete behavior: brand_id / category_id FKs are on delete restrict ; product_id FKs on variants/images are on delete cascade

3.3 Indexing

- products(category_id) , products(brand_id) , categories(parent_id)
- GIN index on products(specs) using jsonb_path_ops

- btree on `product_variants(price_ils)`; GIN/btree on `product_variants(dimensions)` if needed
- Generated `tsvector` column on `products` (name + description, 'simple' config — Hebrew doesn't benefit from English stemming), with GIN index for search
- Index on `product_variants(stock_status)`
- Indexes on `published` columns across brands/categories/products/product_variants (belt-and-braces at this row count)
- Unique indexes: `brands.slug`, `categories.slug`, `products.slug`, `product_variants.sku`, plus the partial unique on `product_images` primary-image

3.4 Row-Level Security

Same pattern as the existing `products` table: RLS enabled on every table; anon/authenticated get SELECT-only on published rows via policy; no public insert/update/delete (catalog managed via Studio using the postgres role, which bypasses RLS); an explicit `grant select` is required per table in addition to the RLS policy (a known PostgreSQL gotcha).

- **brands / categories / products:** `using (published = true)`
- **product_variants:** `using (published = true and exists (select 1 from products p where p.id = product_variants.product_id and p.published = true))` — a variant can be independently hidden but is never shown if the parent product is hidden
- **product_images:** no own published column; `using (exists (...product published...) and (variant_id is null or exists (...variant published...)))` — visibility is fully inherited

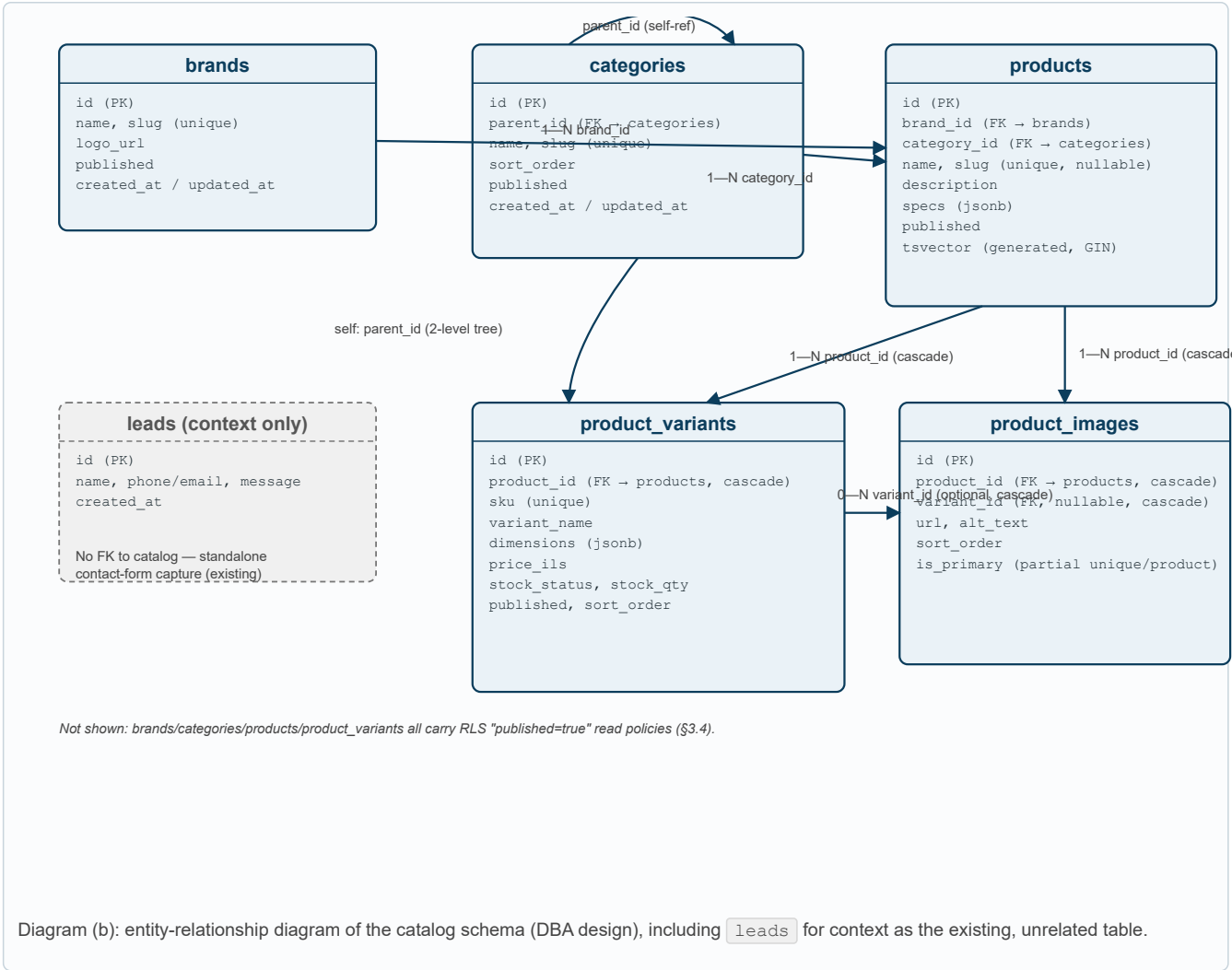
3.5 Search recommendation (DBA, reaffirmed by this design)

NO NEW PAID SERVICE — COST-AVOIDANCE RECOMMENDATION Postgres full-text search (tsvector + GIN), **not** a dedicated search service (Algolia, Meilisearch, Elasticsearch). At 2 brands / dozens of SKUs, Postgres FTS plus the facet indexes above (category/brand/price/stock) is fully sufficient for both relevance search and faceted browse, with no new moving parts, no new operational/security surface, and no recurring cost. If the catalogue later grows to hundreds/thousands of SKUs with heavy real-time typeahead needs, that is a future revisit requiring CIO/CEO/parent decision — not decided here.

3.6 Proportionality notes (intentionally avoided by the DBA)

- No inventory movement/audit trail (the `stock_status` enum matches the order-on-demand model, not a warehouse system)
- No `product_categories` many-to-many (a single `category_id` fits the current catalog shape)
- No nested-set/ltree for categories (plain `parent_id` is enough for a 2-level tree)
- No dedicated search service (see §3.5)
- `specs` as jsonb rather than a full EAV attribute system

The one place the DBA added structure ahead of strictly proven need: `product_variants` as its own table — justified because retrofitting it after real pricing/orders exist would be far more disruptive than building it now.



4. Key Catalogue Features — Summary

Feature	How it's delivered
Browsing / categories	<code>catalog.html</code> reads the (small, 2-level) category tree from <code>categories</code> , rendered as a simple nav/sidebar; selecting a category filters via <code>category_id</code> query param.
Filtering	Brand, category, price range, stock status — PostgREST query-parameter filters (§2.3) against indexed columns; facet counts computed client-side at this scale.
Search	Free-text search box hits the generated <code>tsvector</code> column via PostgREST's <code>fts</code> filter (§2.3, §3.5) — Postgres FTS, no external search service.
Product detail pages	<code>product.html?id=...</code> , fetches one product + its variants + images; variant picker if >1 variant (§2.4).
Images	Self-hosted Supabase Storage bucket, public read, Studio-based upload (§2.5).
Variants (size/color/model)	<code>product_variants</code> rows per product; picker swaps price/stock/dimensions/images (§2.4).
Stock/availability display	<code>stock_status</code> enum mapped to Hebrew labels/colors (§2.4), worded to match the order-on-demand fulfillment model.

5. Open Questions / Risks

5.1 From the DBA (data layer)

1. **Hebrew slugs** — a "slug" column is an English SEO convention that doesn't map cleanly onto Hebrew names. Partially addressed in §2.2 (brands/categories get real slugs now; products/variants route by id/sku until Marketing settles a convention) — final call is a Marketing/CIO decision, not settled here.
2. **specs jsonb key-naming convention** needs to be agreed at the app level (e.g. fixed key sets per category), since jsonb isn't schema-enforced. Recommend Dev and DBA agree a short written convention (e.g. a per-category key list) before the catalog UI is built against it, so the frontend isn't guessing at key names.
3. **Migrating the 7 existing seeded (unpublished) Char-Broil rows** from the old free-text columns (dimensions/burners/features/price_tier) to the new structured shape is its own reviewed migration step for Dev — zero live-traffic risk since nothing is published yet.
4. **Category cycle prevention is intentionally shallow** (blocks only direct self-parenting) — accepted given the 2-level hierarchy.
5. **Backup/restore** — the schema-agnostic `backup-db.ps1` (`pg_dump`) will pick up the new tables automatically, but the restore path has never been tested end-to-end. DBA recommends an actual restore drill once any schema is applied, independent of this catalog design.

5.2 From this design (Solution Architect)

1. **SEO / social-link-preview tradeoff (§2.1)** — client-side-only rendering means shared product links (e.g. via WhatsApp, a very common Israeli sharing channel) won't show a rich preview, since link-preview bots don't execute JS. Flagged as a genuine future decision (Option B, build-time pre-rendering), not solved by this design; revisit once Marketing has an active plan to drive traffic to individual product links.
2. **Facet-count performance at scale** — client-side facet counting is fine at dozens of SKUs; if the catalogue grows substantially, the DBA's suggested materialized view (refreshed on catalog change) is the recommended next step, not a day-one requirement.
3. **Image sizing discipline** — no automatic image optimization exists in the self-hosted stack; relies on twins/Marketing manually sizing images reasonably before upload (§2.5). Worth a one-line note in whatever internal "how to add a product" instructions Dev/CIO eventually write for the twins.
4. **"In stock" wording vs. order-on-demand reality (§2.4)** — flagged for Marketing sign-off on exact Hebrew copy so stock badges don't imply held inventory that doesn't exist.
5. **Studio as the only content-management path** — consistent with the existing approach (no custom admin panel built), but worth the CIO confirming the twins are comfortable inserting rows across five related tables (brands → categories → products → variants → images) via Studio's table editor rather than a single guided form. Not a blocker, just a usability note for whoever trains the twins on it.

6. Recommendations Requiring Sign-off

#	Item	Recommendation	New cost?
1	Frontend pattern	Keep plain HTML/CSS/JS, client-side fetch from PostgREST, no framework, no build step (§2.1, Option A)	None
2	Build-time SEO/link-preview pre-rendering	Defer for now (§2.1, Option B) — revisit once Marketing has an active traffic/sharing plan for product links	None (future option only)
3	Search	Reaffirm DBA's Postgres full-text search recommendation — no dedicated search service (§3.5)	None
4	Images	Use the self-hosted Supabase Storage service already in the stack (§2.5) — new use of an existing component	None new (already-approved stack); indirectly adds to existing EC2 disk estimate
5	URL/routing scheme	Real slugs for brands/categories now; id/sku-based routing for products/variants until Marketing settles a Hebrew slug convention (§2.2)	None
6	Facet-count optimization	No materialized view now — client-side counting at current scale; DBA's materialized-view idea deferred to a future revisit	None
7	External image CDN / dedicated search service	Explicitly rejected for now as disproportionate to current scale/traffic	Avoided

None of the above introduces a new paid third-party service. The only genuine architecture decision point left open for the CIO/CEO/parent is item 2 (build-time pre-rendering) — presented as an either/or, not decided by this document.

Mangalist — Solution Architect design deliverable, produced at the CIO's direction per the parent's 2026-07-05 request. Design/planning only — no code, migrations, or site changes were made as part of this task. Hand-off: CIO reviews this design, then routes any resulting implementation work to Dev (frontend/catalog pages) and DBA (schema migration), with QA verifying once built. This document does not constitute approval of anything in it.